Cloud Hypervisor + GUI — Complete Setup Guide (Corrected)

Goal: Run Firefox browser inside Cloud Hypervisor microVM with GUI via VNC (tightvncserver + XFCE)

Requirements:

- Ubuntu 22.04 LTS or later
- KVM support (Intel VT-x or AMD-V)
- 8GB RAM minimum
- Rust compiler (will install)

> **Note on performance expectations:** Cloud Hypervisor's hypervisor initialisation is ~100–200ms, but a full Ubuntu guest with desktop, cloud-init, and VNC takes **60–120 seconds** on warm boots and **10–20 minutes on first boot** (due to apt package installation). Plan accordingly.

Part 1: System Preparation

Step 1.1: Verify Virtualization Support

```bash
# Check CPU virtualization
egrep -c '(vmx|svm)' /proc/cpuinfo
# Should return number > 0

# Check KVM availability
ls -l /dev/kvm
# Should exist

# Load KVM modules if not loaded
sudo modprobe kvm
sudo modprobe kvm_intel  # OR: sudo modprobe kvm_amd

# Add yourself to the kvm group so cloud-hypervisor can access /dev/kvm without sudo
sudo usermod -aG kvm $USER
# IMPORTANT: log out and back in (or run: newgrp kvm) for group change to take effec
```

Step 1.2: Install Dependencies

bash

```bash
# Update system
sudo apt update && sudo apt upgrade -y

# Install all required packages (host side)
sudo apt install -y \
    build-essential \
    git \
    curl \
    pkg-config \
    libssl-dev \
    libglib2.0-dev \
    libpixman-1-dev \
    libcap-ng-dev \
    qemu-utils \
    cloud-image-utils \
    genisoimage \
    sshpass \
    tigervnc-viewer \
    remmina \
    remmina-plugin-rdp \
    netcat-openbsd \
    nbd-client

# Install Rust (Cloud Hypervisor is written in Rust)
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source $HOME/.cargo/env

# Verify Rust installation
rustc --version
cargo --version
```

Part 2: Build Cloud Hypervisor

Step 2.1: Clone and Compile

```bash
# Create working directory
mkdir -p ~/cloud-hypervisor-setup
cd ~/cloud-hypervisor-setup

# Clone Cloud Hypervisor repository
git clone https://github.com/cloud-hypervisor/cloud-hypervisor.git
cd cloud-hypervisor

# Build (this takes 5-15 minutes)
cargo build --release

# Install binary
sudo cp target/release/cloud-hypervisor /usr/local/bin/
sudo chmod +x /usr/local/bin/cloud-hypervisor

# Verify installation
cloud-hypervisor --version
# Should show: cloud-hypervisor v38.0 or similar

cd ~/cloud-hypervisor-setup
```

Part 3: Create Guest OS Image

Step 3.1: Download Ubuntu Cloud Image

```bash
cd ~/cloud-hypervisor-setup

# Download Ubuntu 22.04 cloud image
wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img

# IMPORTANT: copy FIRST, then resize the copy — preserve the original for reuse
cp jammy-server-cloudimg-amd64.img ubuntu-browser.img
qemu-img resize ubuntu-browser.img +10G

echo "Base image downloaded, copied, and resized"
```

Step 3.2: Extract Kernel and Initrd from Guest Image

> **Why this matters:** Cloud Hypervisor uses direct kernel boot — it needs both a kernel (`vmlinuz`) and an initial ramdisk (`initrd`). You CANNOT use the host's `/boot/vmlinuz`; it lacks the virtio drivers the guest needs and has a different ABI. Always extract from the guest image itself.

```bash
cd ~/cloud-hypervisor-setup

# Load the nbd kernel module
sudo modprobe nbd

# Mount the cloud image as a block device
sudo qemu-nbd --connect=/dev/nbd0 ubuntu-browser.img

# Wait a moment for partitions to appear
sleep 2
sudo partprobe /dev/nbd0

# Mount the root partition (cloud images use partition 1)
sudo mkdir -p /mnt/guest
sudo mount /dev/nbd0p1 /mnt/guest

# Copy the latest kernel and initrd out of the image
GUEST_KERNEL=$(ls /mnt/guest/boot/vmlinuz-* | sort -V | tail -1)
GUEST_INITRD=$(ls /mnt/guest/boot/initrd.img-* | sort -V | tail -1)

cp "$GUEST_KERNEL" ./guest-vmlinuz
cp "$GUEST_INITRD" ./guest-initrd.img

echo "Extracted kernel: $GUEST_KERNEL"
echo "Extracted initrd: $GUEST_INITRD"

# Unmount cleanly
sudo umount /mnt/guest
sudo qemu-nbd --disconnect /dev/nbd0
```

Step 3.3: Create Cloud-Init Configuration

bash

```
cd ~/cloud-hypervisor-setup

# Generate a proper SHA-512 password hash
# (plain_text_passwd is acceptable for dev/lab use)
PASSWD_HASH=$(python3 -c "import crypt,secrets; \
    print(crypt.crypt('browser123', crypt.mksalt(crypt.METHOD_SHA512)))")

echo "Generated hash: $PASSWD_HASH"

# Create user-data file for cloud-init
cat > user-data << EOF
#cloud-config

hostname: browser-isolation

users:
  - name: browser
    sudo: ALL=(ALL) NOPASSWD:ALL
    groups: users, admin, kvm
    home: /home/browser
    shell: /bin/bash
    lock_passwd: false
    passwd: ${PASSWD_HASH}

# Install desktop, VNC server, and Firefox
packages:
  - xfce4
  - xfce4-goodies
  - tightvncserver
  - xserver-xorg-video-dummy
  - firefox
  - net-tools
  - openssh-server
  - inotify-tools

# Write VNC startup script before runcmd runs
write_files:
  - path: /home/browser/.vnc/xstartup
    owner: browser:browser
    permissions: '0755'
    content: |
      #!/bin/bash
      xrdb \$HOME/.Xresources 2>/dev/null || true
      startxfce4 &

  - path: /etc/X11/xorg.conf
    content: |
```

```
      Section "Device"
        Identifier "DummyDevice"
        Driver "dummy"
        VideoRam 256000
      EndSection
      Section "Screen"
        Identifier "DummyScreen"
        Device "DummyDevice"
        DefaultDepth 24
        SubSection "Display"
          Depth 24
          Modes "1920x1080"
        EndSubSection
      EndSection

runcmd:
  - systemctl enable ssh
  - systemctl start ssh
  # Set VNC password non-interactively
  - su -c 'mkdir -p /home/browser/.vnc && echo browser123 | vncpasswd -f > /home/bro
  # Start VNC server on display :1 (port 5901)
  - su -c 'vncserver :1 -geometry 1920x1080 -depth 24' browser

chpasswd:
  list: |
    root:browser123
    browser:browser123
  expire: False

growpart:
  mode: auto
  devices: ['/']
EOF

# Create meta-data
cat > meta-data << 'EOF'
instance-id: browser-isolation-001
local-hostname: browser-isolation
EOF

# Create network-config
# Uses a wildcard match because virtio-net inside Cloud Hypervisor
# is named ens3 or enp1s0 — NOT enp0s3 (which is VirtualBox naming)
cat > network-config << 'EOF'
version: 2
ethernets:
  id0:
    match:
      name: "en*"
    dhcp4: true
```

```
EOF

# Generate the cloud-init seed ISO
cloud-localds -N network-config cloud-init.img user-data meta-data

echo "Cloud-init configuration created"
```

Step 3.4: Generate SSH Key for Secure VM Access

bash

```
cd ~/cloud-hypervisor-setup

# Generate a dedicated key pair — no passwords in scripts
ssh-keygen -t ed25519 -f ~/.ssh/ch_browser -N "" -C "cloud-hypervisor-browser-vm"

# Append the public key to cloud-init user-data
# (re-generate cloud-init.img after this)
SSH_PUBKEY=$(cat ~/.ssh/ch_browser.pub)

cat >> user-data << EOF

ssh_authorized_keys:
  - ${SSH_PUBKEY}
EOF

# Regenerate cloud-init ISO with the updated user-data
cloud-localds -N network-config cloud-init.img user-data meta-data

echo "SSH key injected into cloud-init"
```

Part 4: Set Up Networking

Step 4.1: Create TAP Device

bash

```
cat > ~/cloud-hypervisor-setup/setup-network.sh << 'EOF'
#!/bin/bash
set -e

TAP_IFACE="tap-ch0"
HOST_IP="192.168.100.1"
SUBNET="192.168.100.0/24"

# Create TAP device owned by the current user (not root)
# This allows cloud-hypervisor to open it without running as root
if ! ip link show "$TAP_IFACE" &>/dev/null; then
    sudo ip tuntap add "$TAP_IFACE" mode tap user "$USER" group kvm
    echo "Created TAP device: $TAP_IFACE"
else
    echo "TAP device $TAP_IFACE already exists"
fi

sudo ip addr add "$HOST_IP/24" dev "$TAP_IFACE" 2>/dev/null || true
sudo ip link set "$TAP_IFACE" up

# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1 > /dev/null

# Get the main outbound interface
MAIN_IF=$(ip route show default | awk '{print $5}' | head -1)

# Set up NAT masquerading
sudo iptables -t nat -C POSTROUTING -s "$SUBNET" -o "$MAIN_IF" -j MASQUERADE 2>/dev/
    sudo iptables -t nat -A POSTROUTING -s "$SUBNET" -o "$MAIN_IF" -j MASQUERADE

sudo iptables -C FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT 2>/dev
    sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT

sudo iptables -C FORWARD -i "$TAP_IFACE" -o "$MAIN_IF" -j ACCEPT 2>/dev/null || \
    sudo iptables -A FORWARD -i "$TAP_IFACE" -o "$MAIN_IF" -j ACCEPT

echo "Network configured: $TAP_IFACE at $HOST_IP"
echo "Guest will receive IP via DHCP (typically 192.168.100.2)"
EOF

chmod +x ~/cloud-hypervisor-setup/setup-network.sh

# Run it
~/cloud-hypervisor-setup/setup-network.sh
```

Part 5: Configure Cloud Hypervisor

Step 5.1: Create VM Configuration File

bash

```bash
cd ~/cloud-hypervisor-setup

cat > vm-config.json << EOF
{
  "cpus": {
    "boot_vcpus": 2,
    "max_vcpus": 2
  },
  "memory": {
    "size": 2147483648
  },
  "kernel": {
    "path": "$HOME/cloud-hypervisor-setup/guest-vmlinuz"
  },
  "initramfs": {
    "path": "$HOME/cloud-hypervisor-setup/guest-initrd.img"
  },
  "cmdline": {
    "args": "console=hvc0 root=/dev/vda1 rw quiet splash"
  },
  "disks": [
    {
      "path": "$HOME/cloud-hypervisor-setup/ubuntu-browser.img"
    },
    {
      "path": "$HOME/cloud-hypervisor-setup/cloud-init.img",
      "readonly": true
    }
  ],
  "net": [
    {
      "tap": "tap-ch0",
      "mac": "12:34:56:78:90:ab"
    }
  ],
  "console": {
    "mode": "Tty"
  },
  "serial": {
    "mode": "Null"
  }
}
EOF

echo "VM configuration written to vm-config.json"
```

Step 5.2: Create VNC Launcher Script

> **Design decision:** This guide uses **tightvncserver + XFCE** exclusively. The original guide mixed x11vnc (port 5900) and tightvncserver (port 5901) — this version is consistent throughout. VNC display :1 = port **5901**.

bash

```
cat > ~/cloud-hypervisor-setup/launch-vnc.sh << 'EOF'
#!/bin/bash
set -e

cd ~/cloud-hypervisor-setup

VM_IP="192.168.100.2"
VNC_PORT="5901"
SSH_KEY="$HOME/.ssh/ch_browser"

echo "Starting Cloud Hypervisor with VNC..."

# Ensure network is ready
./setup-network.sh

# Launch Cloud Hypervisor (no sudo needed — user owns the TAP device and is in kvm g
cloud-hypervisor \
    --kernel ./guest-vmlinuz \
    --initramfs ./guest-initrd.img \
    --cmdline "console=hvc0 root=/dev/vda1 rw quiet" \
    --cpus boot=2 \
    --memory size=2G \
    --disk path=ubuntu-browser.img \
    --disk path=cloud-init.img,readonly=on \
    --net tap=tap-ch0,mac=12:34:56:78:90:ab \
    --console tty \
    --serial null &

CH_PID=$!
echo "Cloud Hypervisor started (PID: $CH_PID)"

# Wait for SSH to become available — more reliable than ping
# (network stack is up much earlier than SSH; this confirms cloud-init has run enoug
echo "Waiting for SSH on $VM_IP..."
TIMEOUT=180
ELAPSED=0
while ! nc -z "$VM_IP" 22 2>/dev/null; do
    sleep 2
    ELAPSED=$((ELAPSED + 2))
    if [ $ELAPSED -ge $TIMEOUT ]; then
        echo "Timed out waiting for VM SSH after ${TIMEOUT}s. Check console output."
        kill $CH_PID 2>/dev/null
        exit 1
    fi
    echo "  ...waiting (${ELAPSED}s)"
done
```

```
echo "SSH is up. Waiting for VNC port $VNC_PORT..."
while ! nc -z "$VM_IP" "$VNC_PORT" 2>/dev/null; do
    sleep 2
    ELAPSED=$((ELAPSED + 2))
    echo "  ...waiting for VNC (${ELAPSED}s)"
done

echo "VM ready. Opening VNC viewer..."

# Store host key on first connect, reject mismatches after that
ssh-keyscan -H "$VM_IP" >> ~/.ssh/known_hosts_chvm 2>/dev/null

# Connect with VNC viewer (tightvncserver display :1 = port 5901)
vncviewer "$VM_IP:$VNC_PORT" &

echo "Done. VM PID: $CH_PID"
echo "Press Ctrl+C to shut down the VM"

wait $CH_PID
EOF

chmod +x ~/cloud-hypervisor-setup/launch-vnc.sh
```

Step 5.3: Create RDP Launcher Script (Alternative)

bash

```
cat > ~/cloud-hypervisor-setup/launch-rdp.sh << 'EOF'
#!/bin/bash
set -e

cd ~/cloud-hypervisor-setup

VM_IP="192.168.100.2"

echo "Starting Cloud Hypervisor with RDP (xrdp)..."

./setup-network.sh

cloud-hypervisor \
    --kernel ./guest-vmlinuz \
    --initramfs ./guest-initrd.img \
    --cmdline "console=hvc0 root=/dev/vda1 rw quiet" \
    --cpus boot=2 \
    --memory size=2G \
    --disk path=ubuntu-browser.img \
    --disk path=cloud-init.img,readonly=on \
    --net tap=tap-ch0,mac=12:34:56:78:90:ab \
    --console off \
    --serial null &

CH_PID=$!

echo "Waiting for RDP on $VM_IP:3389..."
ELAPSED=0
while ! nc -z "$VM_IP" 3389 2>/dev/null; do
    sleep 2
    ELAPSED=$((ELAPSED + 2))
    echo "  ...waiting (${ELAPSED}s)"
done

echo "VM ready. Connecting via RDP..."
remmina -c "rdp://browser@$VM_IP" &

echo "VM PID: $CH_PID"
wait $CH_PID
EOF

chmod +x ~/cloud-hypervisor-setup/launch-rdp.sh
```

Part 6: First Boot

Step 6.1: Launch the VM

```bash
cd ~/cloud-hypervisor-setup

# Ensure network is set up
./setup-network.sh

# Launch via VNC
./launch-vnc.sh

# Expected timeline:
#   ~100-200ms  — Cloud Hypervisor hypervisor initialisation
#   ~5-10s      — Guest kernel + initrd boot
#   ~10-20min   — cloud-init first boot (apt installs xfce4, tightvncserver, firefox
#   ~60-120s    — Subsequent boots to VNC-ready state
```

Step 6.2: Verify VNC Connection

```bash
# VNC on display :1 = port 5901 (tightvncserver)
vncviewer 192.168.100.2:5901
# VNC password: browser123

# You should see the XFCE desktop with Firefox available
```

Step 6.3: SSH Access (Key-Based)

```bash
# SSH using the dedicated key generated in Step 3.4
ssh -i ~/.ssh/ch_browser \
    -o UserKnownHostsFile=~/.ssh/known_hosts_chvm \
    browser@192.168.100.2
```

Part 7: Connect to the VM

Step 7.1: VNC Connection

```bash
# From host machine — always use port 5901 (display :1 from tightvncserver)
vncviewer 192.168.100.2:5901
# Password: browser123

# You will see the XFCE desktop with Firefox available in the Applications menu
```

Step 7.2: RDP Connection (Alternative)

```bash
# Connect via RDP (requires xrdp to be installed in the guest via cloud-init)
remmina -c rdp://browser@192.168.100.2
# Password: browser123
```

Part 8: File Monitoring

Step 8.1: Monitor Script (Host Side)

bash

```
cat > ~/cloud-hypervisor-setup/file-monitor.py << 'EOF'
#!/usr/bin/env python3
"""
Monitor file operations in the Cloud Hypervisor VM via SSH.
Requires: pip3 install paramiko
"""

import paramiko
import os
import logging

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s'
)

VM_IP       = "192.168.100.2"
USERNAME    = "browser"
SSH_KEY     = os.path.expanduser("~/.ssh/ch_browser")
KNOWN_HOSTS = os.path.expanduser("~/.ssh/known_hosts_chvm")
WATCH_PATH  = "/home/browser/Downloads"


def monitor_vm():
    ssh = paramiko.SSHClient()

    # Use RejectPolicy with an explicit known_hosts file — never AutoAddPolicy
    ssh.set_missing_host_key_policy(paramiko.RejectPolicy())
    try:
        ssh.load_host_keys(KNOWN_HOSTS)
    except FileNotFoundError:
        logging.error(
            f"known_hosts file not found: {KNOWN_HOSTS}\n"
            "Run: ssh-keyscan -H 192.168.100.2 >> ~/.ssh/known_hosts_chvm"
        )
        return

    try:
        ssh.connect(
            VM_IP,
            username=USERNAME,
            key_filename=SSH_KEY,
            look_for_keys=False,
            allow_agent=False,
        )
        logging.info(f"Connected to VM: {VM_IP}")
```

```
        cmd = (
            f"inotifywait -m {WATCH_PATH} "
            f"-e create -e modify -e delete --format '%T %e %f' --timefmt '%H:%M:%S'"
        )
        _, stdout, _ = ssh.exec_command(cmd)

        for line in stdout:
            logging.warning(f"FILE EVENT: {line.strip()}")

    except KeyboardInterrupt:
        logging.info("Monitoring stopped by user")
    except paramiko.AuthenticationException:
        logging.error("SSH authentication failed — check your key pair")
    except paramiko.SSHException as e:
        logging.error(f"SSH error: {e}")
    finally:
        ssh.close()


if __name__ == "__main__":
    # Install dependency: pip3 install paramiko
    monitor_vm()
EOF

chmod +x ~/cloud-hypervisor-setup/file-monitor.py
pip3 install paramiko
```

Part 9: Desktop Integration

Step 9.1: Create Desktop Launcher

bash

```
cat > ~/.local/share/applications/cloud-hypervisor-browser.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=Isolated Browser (Cloud Hypervisor)
Comment=Launch browser in Cloud Hypervisor microVM
Exec=$HOME/cloud-hypervisor-setup/launch-vnc.sh
Icon=web-browser
Terminal=true
Categories=Network;WebBrowser;Security;
EOF

update-desktop-database ~/.local/share/applications/
echo "Desktop launcher created"
```

Part 10: Performance Benchmarking

Step 10.1: Boot Time Test

> **Note:** This script measures time-to-SSH-ready, not time-to-browser. SSH becoming available indicates the network stack and openssh-server are up, which is a meaningful readiness signal. Full desktop + VNC readiness will be an additional 15–30 seconds beyond SSH.

bash

```bash
cat > ~/cloud-hypervisor-setup/benchmark.sh << 'EOF'
#!/bin/bash
set -e

cd ~/cloud-hypervisor-setup

VM_IP="192.168.100.2"

echo "Benchmarking Cloud Hypervisor boot time..."

./setup-network.sh

START_MS=$(date +%s%3N)

cloud-hypervisor \
    --kernel ./guest-vmlinuz \
    --initramfs ./guest-initrd.img \
    --cmdline "console=hvc0 root=/dev/vda1 rw quiet" \
    --cpus boot=2 \
    --memory size=2G \
    --disk path=ubuntu-browser.img \
    --net tap=tap-ch0,mac=12:34:56:78:90:ab \
    --console off \
    --serial null &

CH_PID=$!

# Wait for SSH — far more meaningful than ping (network can be up before SSH)
echo "Waiting for SSH on $VM_IP..."
while ! nc -z "$VM_IP" 22 2>/dev/null; do
    sleep 0.5
done

SSH_MS=$(date +%s%3N)

# Also measure VNC readiness
while ! nc -z "$VM_IP" 5901 2>/dev/null; do
    sleep 0.5
done

VNC_MS=$(date +%s%3N)

echo ""
echo "--- Results ---"
echo "Time to SSH ready:  $((SSH_MS - START_MS)) ms"
echo "Time to VNC ready:  $((VNC_MS - START_MS)) ms"
echo ""
```

```
echo "Expected ranges (subsequent boots, warm disk cache):"
echo "  SSH:  30,000 - 90,000 ms   (30-90 seconds)"
echo "  VNC:  45,000 - 120,000 ms  (45-120 seconds)"

kill $CH_PID 2>/dev/null
wait $CH_PID 2>/dev/null
echo "VM stopped."
EOF

chmod +x ~/cloud-hypervisor-setup/benchmark.sh
```

Configuration Files Summary

```
cloud-hypervisor-setup/
├── jammy-server-cloudimg-amd64.img   # original base image (untouched)
├── ubuntu-browser.img                # working VM disk (resized copy)
├── cloud-init.img                    # cloud-init seed ISO
├── guest-vmlinuz                     # kernel extracted from guest image
├── guest-initrd.img                  # initrd extracted from guest image
├── vm-config.json                    # VM configuration (JSON format)
├── user-data                         # cloud-init user data
├── meta-data                         # cloud-init metadata
├── network-config                    # cloud-init network config
├── setup-network.sh                  # TAP + NAT setup
├── launch-vnc.sh                     # VNC launcher (primary method)
├── launch-rdp.sh                     # RDP launcher (alternative)
├── file-monitor.py                   # file event monitoring via SSH
└── benchmark.sh                      # boot time benchmark

~/.ssh/
├── ch_browser                        # private key for VM SSH access
├── ch_browser.pub                    # public key (injected via cloud-init)
└── known_hosts_chvm                  # VM-specific known hosts (no global
pollution)
```

Quick Start Commands

```bash
# 1. One-time: set up networking
cd ~/cloud-hypervisor-setup
./setup-network.sh

# 2. Launch with VNC (primary method — tightvncserver on port 5901)
./launch-vnc.sh

# 3. OR launch with RDP (alternative)
./launch-rdp.sh

# 4. Monitor file events in VM (separate terminal)
python3 file-monitor.py

# 5. Benchmark boot time
./benchmark.sh
```

Realistic Performance Expectations

Metric	Value
Hypervisor init	~100–200 ms
Guest kernel + initrd load	~3–8 seconds
First boot (cloud-init + apt)	10–20 minutes
Subsequent boots to SSH ready	30–90 seconds
Subsequent boots to VNC ready	45–120 seconds
RAM usage (guest + host)	~2–2.5 GB

Advantages & Disadvantages

Advantages:

- Own guest kernel (hardware-level isolation from host)
- Better display support than Firecracker
- GPU passthrough available via virtio-gpu
- Active upstream development
- Security model comparable to Firecracker

Disadvantages:

- Smaller community than QEMU/KVM
- Direct kernel boot requires kernel/initrd management
- Still requires VNC/RDP for GUI access
- Linux host only

Troubleshooting

VM won't boot — kernel panic:

```bash
# Most likely cause: missing initrd or wrong kernel
# Verify your extracted files exist and are non-zero:
ls -lh ~/cloud-hypervisor-setup/guest-vmlinuz
ls -lh ~/cloud-hypervisor-setup/guest-initrd.img

# Re-run the extraction from Step 3.2 if files are missing or 0 bytes
```

Permission denied on /dev/kvm or TAP device:

```bash
# Ensure you are in the kvm group
groups | grep kvm
# If not: sudo usermod -aG kvm $USER — then log out and back in

# Verify TAP device ownership
ip link show tap-ch0
# Should show: ... group kvm
```

Can't connect via VNC (connection refused):

```bash
# SSH into VM and check VNC status
ssh -i ~/.ssh/ch_browser \
    -o UserKnownHostsFile=~/.ssh/known_hosts_chvm \
    browser@192.168.100.2

# Inside VM:
ps aux | grep vnc           # confirm tightvncserver is running
ss -tulpn | grep 5901       # confirm it's listening on port 5901

# Restart VNC if needed:
vncserver -kill :1
vncserver :1 -geometry 1920x1080 -depth 24
```

SSH host key mismatch (re-provisioned VM):

```bash
# Clear the old key and re-scan
ssh-keygen -R 192.168.100.2 -f ~/.ssh/known_hosts_chvm
ssh-keyscan -H 192.168.100.2 >> ~/.ssh/known_hosts_chvm
```

cloud-init not running (packages not installed):

```bash
# SSH in and check cloud-init status
ssh -i ~/.ssh/ch_browser browser@192.168.100.2
sudo cloud-init status
sudo cat /var/log/cloud-init-output.log | tail -50
```

This completes the corrected Cloud Hypervisor setup guide.